

# Instruction Set Architecture II

Michael C. Hackett

Associate Professor, Computer Science



# Lecture Topics

- Instruction Set Architectures
  - Zero-Address
  - One-Address
  - Two-Address
  - Three-Address
- Pseudo Operations
- Addressing Modes
- Complex Instruction Set Computers (CISC)
- Reduced Instruction Set Computers (RISC)
- RISC-V

# Instruction Set Architectures

- A *zero-address architecture* uses a data structure called a stack.
  - The operands of any operation are the stack's top two values
- Using pseudocode (no particular programming language), we'll demonstrate the calculation of  $5+(4-1)$  in a zero-address architecture

# Instruction Set Architectures

Pushes 1 and 4 onto the stack

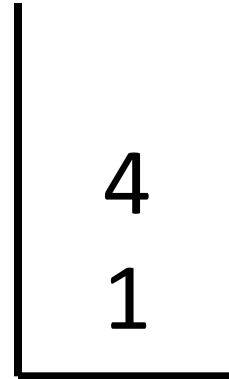
**push 1**

**push 4**

**sub**

**push 5**

**add**



Stack

# Instruction Set Architectures

Takes the two values on top of the stack, subtracts, and puts the result in the stack

**push 1**

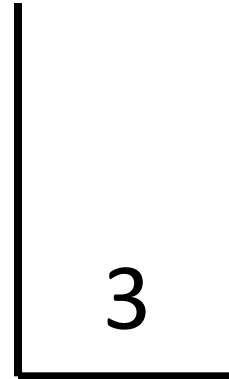
**push 4**

**sub**

$4 - 1 = 3$

**push 5**

**add**



# Instruction Set Architectures

Pushes 5 onto the top of the stack

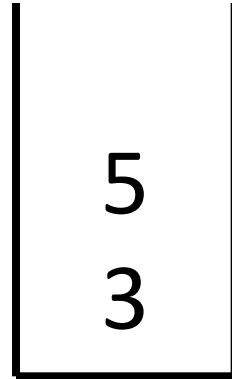
push 1

push 4

sub

push 5

add



# Instruction Set Architectures

Takes the two values on top of the stack, adds, and puts the result in the stack

**push 1**

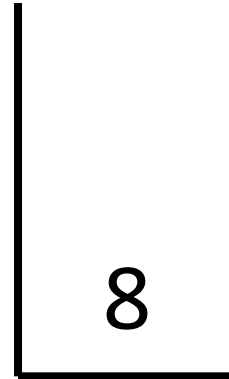
**push 4**

**sub**

**push 5**

**add**

$5 + 3 = 8$



# Instruction Set Architectures

- A *one-address architecture* allows each instruction to use one memory address.
- The CPU uses a special register as an accumulator
  - This register is both an operation's left operand **and** result
- Using pseudocode (no particular programming language), we'll demonstrate the calculation of  $5+(4-1)$  in a one-address architecture

# Instruction Set Architectures

|               |                                  |
|---------------|----------------------------------|
| <b>clear</b>  | <b>#Accumulator = 0</b>          |
| <b>add 1</b>  | <b>#Accumulator = 0 + 1 = 1</b>  |
| <b>negate</b> | <b>#Accumulator = -1</b>         |
| <b>add 4</b>  | <b>#Accumulator = -1 + 4 = 3</b> |
| <b>add 5</b>  | <b>#Accumulator = 3 + 5 = 8</b>  |

# Instruction Set Architectures

- A *two-address architecture* allows each instruction to use two operands.
- The first operand is both the first operand and the register where the result is stored
- Using pseudocode (no particular programming language), we'll demonstrate the calculation of  $5+(4-1)$  in a two-address architecture

# Instruction Set Architectures

**load r1, 5**

**load r2, 4**

**load r3, 1**

**sub r2, r3        #r2 = r2 - r3**

**add r1, r2        #r1 = r1 + r2**

# Instruction Set Architectures

- A *three-address architecture* allows each instruction to use three operands- the left and right operands and the where to store the result.
- MIPS is a three-address architecture
  - As is RISC-V, which will be discussed at the end of the lecture
- The calculation of  $5+(4-1)$  in MIPS:  

|                           |                          |
|---------------------------|--------------------------|
| <b>addi \$t1, \$0, 4</b>  | <b>#\$t1 = 0 + 4 = 4</b> |
| <b>subi \$t1, \$t1, 1</b> | <b>#\$t1 = 4 - 1 = 3</b> |
| <b>addi \$t0, \$t1, 5</b> | <b>#\$t0 = 3 + 5 = 8</b> |

# Instruction Formats

- An **instruction format** defines the arrangement and organization of the bits that form each instruction.
  - Each instruction is 32 bits in MIPS (same as its word size)
- This binary version of instructions is called the **machine language**
  - The assembler converts assembly instructions to machine language
  - The machine language is the actual instructions that are given to and understood by the CPU

# Addressing Modes

- An **addressing mode** specifies the way in which data is used via memory addresses.
  - An ISA may allow the use of multiple addressing modes in the design of its instructions.
  - **Direct addressing** is when an instruction uses the absolute address of an operand.
    - Unconditional jumps in MIPS use direct addressing (though the 26 bits are padded with zeroes to form a 32-bit address)
- j **label**

# Addressing Modes

- **Register addressing** (a form of direct addressing) uses the data stored in registers as operands; no memory addresses needed
  - Instructions like **add** use register addressing  
**add \$t0, \$t1, \$t2**
- **Immediate addressing** (also a form of direct addressing) is when an operand is part of the instruction
  - Immediate instructions in MIPS, like **addi**, use immediate addressing  
**addi \$t0, \$t1, 6**

# Addressing Modes

- **Indirect addressing** is when an instruction accesses an address of memory, which contains the address of the instruction's operand
- **Base addressing** (a form of indirect addressing) is when we add a constant (offset) to an address stored in a register
  - Instructions like **lw** use base addressing  
`lw $t0, 16($a0)`
- **PC-relative addressing** (also a form of indirect addressing) is when an address is formed by adding the program counter with a constant
  - Branch instructions in MIPS, like **bne**, use PC-relative addressing  
`bne $t0, $t1, label/offset`

# Addressing Modes

- An instruction set with instructions that use any supported addressing mode is called an **orthogonal** instruction set.
  - For example, an **add** instruction for direct addressing, register addressing, base addressing, etc.
  - Adds much more complexity to the design of the processor
- **Complex Instruction Set Computers (CISC, “sisk”)**, are processors with a large number of instructions that are more like the instructions of a high-level programming language.
  - The main goal of a CISC architecture is to limit the number of lines of assembly code.
    - Goes hand-in-hand with an orthogonal instruction set

# Complex Instruction Set Computers

- By reducing the number of lines of assembly code this achieves faster compile time of a high-level language to machine language
- This makes it easier to write a compiler for the processor, thus easier to support more high-level languages
  - It also requires less RAM for storing instructions

# Complex Instruction Set Computers

- However, all of the complex instructions must be built into the hardware
  - Most instructions are executed in multiple CPU clock cycles
    - We'll discuss the CPU clock and clock cycles in a later lecture
- Instructions primarily operate directly on memory instead of using registers
  - Register access is faster than main memory access
- CISC architecture and orthogonal instruction sets were superseded by Reduced Instruction Set Computers.
  - CISC got its name retroactively
  - Intel's x86 architecture is basically the only CISC architecture still used in modern processors

# Reduced Instruction Set Computers

- **Reduced Instruction Set Computers (RISC, “risk”)** are processors designed with a limited, yet optimized instruction set.
  - As opposed to the broad and specialized instructions found in a CISC architecture
- Processors with a RISC architecture have:
  - Simple, fixed length instructions that are executed in one CPU clock cycle
  - Few, simple data types
  - Simple addressing modes
  - Large number of general purpose registers

# Reduced Instruction Set Computers

- MIPS is a RISC architecture, as is ARM (Advanced RISC Machine)
- RISC also has the benefit of *pipelining*, which processes instructions more efficiently by executing instructions simultaneously
  - We'll discuss pipelining in a later lecture
  - Pipelining gives RISC the performance advantage over CISC

# RISC-V

- RISC-V (“*risk five*”) is an open standard ISA based on the philosophies of RISC architecture
  - Created by the University of California, Berkeley
- RISC-V is a free and open-source ISA
  - Most ISAs (like ARM and Intel’s x86 and x64) are patented and copyrighted
    - MIPS was proprietary but is now open source
  - Chips designers must pay royalties to use proprietary ISAs in their chip designs

# RISC-V

- See the RISC-V Reference Card posted in Canvas as a reference for this lecture

## Free & Open RISC-V Reference Card ①

| Base Integer Instructions: RV32I, RV64I, and RV128I |                        |     |                   |                        | RV Privileged Instructions         |                               |                   |
|-----------------------------------------------------|------------------------|-----|-------------------|------------------------|------------------------------------|-------------------------------|-------------------|
| Category                                            | Name                   | Fmt | RV32I Base        | +RV{64,128}            | Category                           | Name                          | RV mnemonic       |
| <b>Loads</b>                                        | Load Byte              | I   | LB rd,rs1,imm     |                        | <b>CSR Access</b>                  | Atomic R/W                    | CSRRW rd,csr,rs1  |
|                                                     | Load Halfword          | I   | LH rd,rs1,imm     |                        |                                    | Atomic Read & Set Bit         | CSRRS rd,csr,rs1  |
|                                                     | Load Word              | I   | LW rd,rs1,imm     | L{D Q} rd,rs1,imm      |                                    | Atomic Read & Clear Bit       | CSRRC rd,csr,rs1  |
|                                                     | Load Byte Unsigned     | I   | LBU rd,rs1,imm    |                        |                                    | Atomic R/W Imm                | CSRRWI rd,csr,imm |
|                                                     | Load Half Unsigned     | I   | LHU rd,rs1,imm    | L{W D}U rd,rs1,imm     |                                    | Atomic Read & Set Bit Imm     | CSRRSI rd,csr,imm |
| <b>Stores</b>                                       | Store Byte             | S   | SB rs1,rs2,imm    |                        | <b>Change Level</b>                | Env. Call                     | ECALL             |
|                                                     | Store Halfword         | S   | SH rs1,rs2,imm    |                        |                                    | Environment Breakpoint        | EBREAK            |
|                                                     | Store Word             | S   | SW rs1,rs2,imm    | S{D Q} rs1,rs2,imm     |                                    | Environment Return            | ERET              |
| <b>Shifts</b>                                       | Shift Left             | R   | SLL rd,rs1,rs2    | SLL{W D} rd,rs1,rs2    | <b>Trap Redirect to Supervisor</b> |                               | MRTS              |
|                                                     | Shift Left Immediate   | I   | SLLI rd,rs1,shamt | SLLI{W D} rd,rs1,shamt |                                    | Redirect Trap to Hypervisor   | MRTS              |
|                                                     | Shift Right            | R   | SRL rd,rs1,rs2    | SRL{W D} rd,rs1,rs2    |                                    | Hypervisor Trap to Supervisor | HRTS              |
|                                                     | Shift Right Immediate  | I   | SRLI rd,rs1,shamt | SRLI{W D} rd,rs1,shamt | <b>Interrupt</b>                   | Wait for Interrupt            | WFI               |
|                                                     | Shift Right Arithmetic | R   | SRA rd,rs1,rs2    | SRA{W D} rd,rs1,rs2    |                                    | Supervisor FENCE              | SFENCE.VM rs1     |
| <b>Arithmetic</b>                                   | ADD                    | R   | ADD rd,rs1,rs2    | ADD{W D} rd,rs1,rs2    |                                    |                               |                   |
|                                                     | ADD Immediate          | I   | ADDI rd,rs1,imm   | ADDI{W D} rd,rs1,imm   |                                    |                               |                   |

# RISC-V

- Many conventions you've seen in MIPS are also found in RISC-V
  - You'll notice the similarities between the registers in RISC-V and MIPS

| <b><i>RISC-V Calling Convention</i></b> |          |        |                                  |
|-----------------------------------------|----------|--------|----------------------------------|
| Register                                | ABI Name | Saver  | Description                      |
| x0                                      | zero     | ---    | Hard-wired zero                  |
| x1                                      | ra       | Caller | Return address                   |
| x2                                      | sp       | Callee | Stack pointer                    |
| x3                                      | gp       | ---    | Global pointer                   |
| x4                                      | tp       | ---    | Thread pointer                   |
| x5-7                                    | t0-2     | Caller | Temporaries                      |
| x8                                      | s0/fp    | Callee | Saved register/frame pointer     |
| x9                                      | s1       | Callee | Saved register                   |
| x10-11                                  | a0-1     | Caller | Function arguments/return values |
| x12-17                                  | a2-7     | Caller | Function arguments               |
| x18-27                                  | s2-11    | Callee | Saved registers                  |
| x28-31                                  | t3-t6    | Caller | Temporaries                      |
| f0-7                                    | ft0-7    | Caller | FP temporaries                   |
| f8-9                                    | fs0-1    | Callee | FP saved registers               |
| f10-11                                  | fa0-1    | Caller | FP arguments/return values       |
| f12-17                                  | fa2-7    | Caller | FP arguments                     |
| f18-27                                  | fs2-11   | Callee | FP saved registers               |
| f28-31                                  | ft8-11   | Caller | FP temporaries                   |

| <b>REGISTER NAME, NUMBER, USE, CALL CONVENTION</b> |        |                                                       |                          |
|----------------------------------------------------|--------|-------------------------------------------------------|--------------------------|
| NAME                                               | NUMBER | USE                                                   | PRESERVED ACROSS A CALL? |
| \$zero                                             | 0      | The Constant Value 0                                  | N.A.                     |
| \$at                                               | 1      | Assembler Temporary                                   | No                       |
| \$v0-\$v1                                          | 2-3    | Values for Function Results and Expression Evaluation | No                       |
| \$a0-\$a3                                          | 4-7    | Arguments                                             | No                       |
| \$t0-\$t7                                          | 8-15   | Temporaries                                           | No                       |
| \$s0-\$s7                                          | 16-23  | Saved Temporaries                                     | Yes                      |
| \$t8-\$t9                                          | 24-25  | Temporaries                                           | No                       |
| \$k0-\$k1                                          | 26-27  | Reserved for OS Kernel                                | No                       |
| \$gp                                               | 28     | Global Pointer                                        | Yes                      |
| \$sp                                               | 29     | Stack Pointer                                         | Yes                      |
| \$fp                                               | 30     | Frame Pointer                                         | Yes                      |
| \$ra                                               | 31     | Return Address                                        | Yes                      |

# RISC-V

- You'll recognize a number of MIPS instructions that also appear in RISC-V

| <i>Category</i>   | <i>Name</i>            | <i>Fmt</i> | <i>RV32I Base</i> |              |
|-------------------|------------------------|------------|-------------------|--------------|
| <b>Loads</b>      | Load Byte              | I          | LB                | rd,rs1,imm   |
|                   | Load Halfword          | I          | LH                | rd,rs1,imm   |
|                   | Load Word              | I          | LW                | rd,rs1,imm   |
|                   | Load Byte Unsigned     | I          | LBU               | rd,rs1,imm   |
|                   | Load Half Unsigned     | I          | LHU               | rd,rs1,imm   |
| <b>Stores</b>     | Store Byte             | S          | SB                | rs1,rs2,imm  |
|                   | Store Halfword         | S          | SH                | rs1,rs2,imm  |
|                   | Store Word             | S          | SW                | rs1,rs2,imm  |
| <b>Shifts</b>     | Shift Left             | R          | SLL               | rd,rs1,rs2   |
|                   | Shift Left Immediate   | I          | SLLI              | rd,rs1,shamt |
|                   | Shift Right            | R          | SRL               | rd,rs1,rs2   |
|                   | Shift Right Immediate  | I          | SRLI              | rd,rs1,shamt |
|                   | Shift Right Arithmetic | R          | SRA               | rd,rs1,rs2   |
|                   | Shift Right Arith Imm  | I          | SRAI              | rd,rs1,shamt |
| <b>Arithmetic</b> | ADD                    | R          | ADD               | rd,rs1,rs2   |
|                   | ADD Immediate          | I          | ADDI              | rd,rs1,imm   |
|                   | SUBtract               | R          | SUB               | rd,rs1,rs2   |
|                   | Load Upper Imm         | U          | LUI               | rd,imm       |
|                   | Add Upper Imm to PC    | U          | AUIPC             | rd,imm       |
| <b>Logical</b>    | XOR                    | R          | XOR               | rd,rs1,rs2   |
|                   | XOR Immediate          | I          | XORI              | rd,rs1,imm   |
|                   | OR                     | R          | OR                | rd,rs1,rs2   |
|                   | OR Immediate           | I          | ORI               | rd,rs1,imm   |
|                   | AND                    | R          | AND               | rd,rs1,rs2   |
|                   | AND Immediate          | I          | ANDI              | rd,rs1,imm   |
| <b>Compare</b>    | Set <                  | R          | SLT               | rd,rs1,rs2   |
|                   | Set < Immediate        | I          | SLTI              | rd,rs1,imm   |
|                   | Set < Unsigned         | R          | SLTU              | rd,rs1,rs2   |
|                   | Set < Imm Unsigned     | I          | SLTIU             | rd,rs1,imm   |
| <b>Branches</b>   | Branch =               | SB         | BEQ               | rs1,rs2,imm  |
|                   | Branch ≠               | SB         | BNE               | rs1,rs2,imm  |
|                   | Branch <               | SB         | BLT               | rs1,rs2,imm  |
|                   | Branch ≥               | SB         | BGE               | rs1,rs2,imm  |
|                   | Branch < Unsigned      | SB         | BLTU              | rs1,rs2,imm  |
|                   | Branch ≥ Unsigned      | SB         | BGEU              | rs1,rs2,imm  |

# RISC-V

- RISC-V has more instruction formats than MIPS

32-bit Instruction Formats

Register Format

Immediate Format

Storage Format

Branch Format

Upper-Immediate Format

Jump Format

R

I

S

SB

U

UJ

31302524212019151412118760

funct7rs2rs1funct3rdopcode

imm[11:0]rs1funct3rdopcode

imm[11:5]rs2rs1funct3imm[4:0]opcode

imm[12]imm[10:5]rs2rs1funct3imm[4:1]imm[11]opcode

imm[31:12]rdopcode

imm[20]imm[10:1]imm[11]imm[19:12]rdopcode

# RISC-V

- Boston University's online RISC-V simulator (BRISC-V) is an easy way to practice with the basics of RISC-V
  - A link to the simulator is posted in Canvas
- You can also upload C source code and it will convert the program to RISC-V assembly.
  - This can give you a good understanding of the compile process and see how high-level instructions become low-level instructions.

# RISC-V

BRISC-V Home

BRISC-V Simulator

Manual & Examples



C source



```
1 int fib(int n) {
2     if (n <= 1) {
3         return n;
4     } else {
5         return fib(n-1)+fib(n-2);
6     }
7 }
8
9 int return_function (int result) {
10     return result;
11 }
12
13 int main(){
14     int n = 9;
15     int result = return_function (fib(n));
16     return result;
17 }
```

RISC-V Assembly



```
0      addi    zero,zero,0
kernel:
1      addi    sp,zero,1536
2      call    main
3      addi    zero,zero,0
4      mv      s1,a0
5      addi    zero,zero,0
6      addi    zero,zero,0
7      auipc   ra,0x0
8      jalr    ra,0(ra)
9      addi    zero,zero,0
10     addi    zero,zero,0
    .file "source.c"
    .option nopic
    .text
    .align 2
    .globl fib
    .type fib, @function
fib:
11     addi    sp,sp,-32
12     sw      ra,28(sp)
13     sw      s0,24(sp)
14     sw      s1,20(sp)
15     addi    s0,sp,32
16     sw      a0,-20(s0)
17     lw      a4,-20(s0)
18     li      a5,1
19     bgt     a4,a5,.L2
20     lw      a5,-20(s0)
```

Registers Memory

| Register |      | Value | Register |      | Value |
|----------|------|-------|----------|------|-------|
| zero     | [0]  | 0     | ra       | [1]  | 0     |
| sp       | [2]  | 0     | gp       | [3]  | 0     |
| tp       | [4]  | 0     | t0       | [5]  | 0     |
| t1       | [6]  | 0     | t2       | [7]  | 0     |
| s0/fp    | [8]  | 0     | s1       | [9]  | 0     |
| a0       | [10] | 0     | a1       | [11] | 0     |
| a2       | [12] | 0     | a3       | [13] | 0     |
| a4       | [14] | 0     | a5       | [15] | 0     |
| a6       | [16] | 0     | a7       | [17] | 0     |
| s2       | [18] | 0     | s3       | [19] | 0     |
| s4       | [20] | 0     | s5       | [21] | 0     |
| s6       | [22] | 0     | s7       | [23] | 0     |
| s8       | [24] | 0     | s9       | [25] | 0     |
| s10      | [26] | 0     | s11      | [27] | 0     |
| t3       | [28] | 0     | t4       | [29] | 0     |
| t5       | [30] | 0     | t6       | [31] | 0     |

Console

```
Successfully compiled.
***** Parser Output *****
Parsing successful!
```

Instruction breakdown

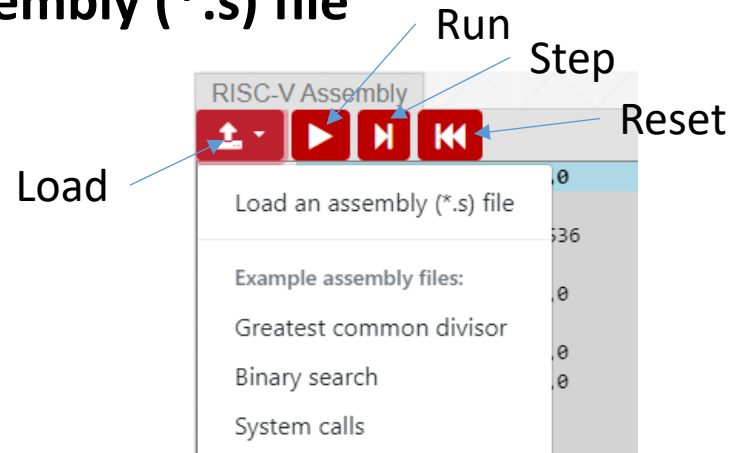
|              |    |       |    |        |    |       |   |         |   |
|--------------|----|-------|----|--------|----|-------|---|---------|---|
| 31           | 20 | 19    | 15 | 14     | 12 | 11    | 7 | 6       | 0 |
| imm          |    | rs1   |    | funct3 |    | rd    |   | opcode  |   |
| 000000000000 |    | 00000 |    | 000    |    | 00000 |   | 0010011 |   |

# RISC-V

- The Module Download contains sample RISC-V assembly code.
  - RISC-V source code files end with the .s extension
- You are encouraged to use these sample programs for exploring RISC-V further.

# RISC-V

- To load the RISC-V assembly code, click the Upload button.
  - Select **Load an assembly (\*.s) file**



- The Run button runs the program
- The Step button executes the program instruction by instruction
- The Reset button resets the program

# RISC-V

- You cannot edit the uploaded assembly file.
  - Make any changes to the assembly file in a text editor, save the changes, and re-upload the modified assembly file.
- The simulator is limited in its capabilities.
  - Some common RISC-V pseudo-ops are not supported
  - Floating point numbers are not supported